

Mission Control + OpenClaw Runbook (Mac mini)

Owner: Brad | Maintainer: Zed | Scope: Day-2 operations (OpenClaw + MissionControlUpdated already installed)

TL;DR (Quick Start)

Use these in order when you want the whole system up quickly.

Start OpenClaw Gateway

```
openclaw gateway status
openclaw gateway start
openclaw gateway status
```

Start Mission Control

```
cd ~/MissionControlUpdated
./scripts/setup.sh
npm run dev
```

Open the dashboard

- Open the URL printed in Terminal (usually `http://127.0.0.1:3000`).

Primary workflow

- Mission Control → Tasks → create task → Dispatch → move status

If Mission Control errors (Next.js ENOENT)

```
cd ~/MissionControlUpdated
rm -rf .next
npm run dev
```

1) What this system is

Components

- **OpenClaw:** local agent runtime + Gateway.
- **Mission Control:** local dashboard for Operations, Tasks, Content, Memory, Team.
- **Dispatch:** sends a task to OpenClaw via local system event (with cooldown).

Security model

- Local-only (binds to 127.0.0.1 by default).
- No public exposure.
- OpenClaw Control UI may require a Gateway token.

2) Where everything lives

Mission Control repo

```
cd ~/MissionControlUpdated  
pwd
```

OpenClaw Control UI URL

```
openclaw status
```

Look for: "Dashboard: <http://127.0.0.1:18789/>" (example).

3) Start, stop, restart (common operations)

A) Mission Control

Start (recommended)

```
cd ~/MissionControlUpdated  
./scripts/setup.sh  
npm run dev
```

Start (fast)

```
cd ~/MissionControlUpdated  
npm run dev
```

Stop

- In the terminal where Mission Control is running: press Ctrl+C

B) OpenClaw Gateway

Check status

```
openclaw gateway status
```

Start

```
openclaw gateway start  
openclaw gateway status
```

Restart (when stuck)

```
openclaw gateway restart  
openclaw gateway status
```

4) Using Mission Control (how to actually work)

Operations tab

- Check Mission Control health.
- Confirm OpenClaw Gateway is reachable and running.

Tasks tab (primary workflow)

- Create tasks.
- Move tasks across: Backlog → In Progress → Blocked → Done.
- Dispatch: click Dispatch on a task card (cooldown prevents spam).

Marketing and Research tabs

- Phase 1 placeholders that route into Content Pipeline and Memory.

5) OpenClaw command cheat sheet

Core health

```
openclaw status
openclaw gateway status
openclaw health
```

Logs

```
openclaw logs
openclaw logs --follow
```

Sessions

```
openclaw sessions
openclaw sessions --json
```

Security audits

```
openclaw security audit
openclaw security audit --deep
```

Cron (scheduled jobs)

```
openclaw cron list
openclaw cron runs <jobId>
openclaw cron run <jobId>
```

6) Troubleshooting (most common issues)

Issue A: "Unauthorized / gateway token missing"

Fix:

1. Get the dashboard URL: `openclaw status`
2. Open that URL (example: `http://127.0.0.1:18789/`)
3. Settings → copy Gateway token → paste where prompted

Issue B: Next.js ENOENT about `.next/server`

Fix:

```
cd ~/MissionControlUpdated
rm -rf .next
npm run dev
```

Issue C: OpenClaw fails due to Node version

```
node -v
which node
which -a node
```

OpenClaw requires Node `>= 22.12.0`.

7) What to send Zed if something breaks

Copy/paste outputs of:

```
openclaw status
openclaw gateway status
openclaw logs
```

Also include which Mission Control URL/port you were using and which page failed.